service in ubuntu all the things

Here's a **complete, beginner → advanced guide to services in Ubuntu** (using **systemd**, which Ubuntu uses by default).

I'll cover everything step-by-step:

✅ What is a service
✅ systemd basics
✅ service commands
✅ systemctl commands (important)
✅ service file structure
✅ create your own service
✅ auto start on boot
✅ logs & debugging
✅ advanced features
✅ real examples

🔹 1. What is a Service?

A **service (daemon)** is a background process that:

- runs automatically

- doesn't need user interaction
- starts at boot

Examples:

- nginx (web server)
- mysql (database)
- ssh (remote login)
- docker
- cron

🔹 2. Ubuntu uses systemd

Modern Ubuntu uses:

👉 **systemd** (NOT init.d anymore)

Main tool:

```nginx
systemctl
```

Copy code

Old:

```swift
service
/etc/init.d/
```

Copy code

👉 You should use **systemctl**, not service.

🔹 3. Basic Service Commands (Daily Use)

List services

bash

```bash
systemctl list-units --type=service
```

Copy code

Only running:

arduino

```arduino
systemctl list-units --type=service --state=running
```

Copy code

All services:

bash

```bash
systemctl list-unit-files --type=service
```

Copy code

Start service

bash

```bash
sudo systemctl start nginx
```

Copy code

Stop service

arduino

```
sudo systemctl stop nginx
```

Copy code

Restart

bash

```bash
sudo systemctl restart nginx
```

Copy code

Reload config (no downtime)

bash

```bash
sudo systemctl reload nginx
```

Copy code

Check status

lua

```lua
systemctl status nginx
```

Copy code

Shows:

- running or not
- PID
- logs
- errors

Enable auto-start at boot

bash

```bash
sudo systemctl enable nginx
```

Copy code

Disable auto-start

bash

```bash
sudo systemctl disable nginx
```

Copy code

Check if enabled

csharp

```csharp
systemctl is-enabled nginx
```

Copy code

Kill service

bash

```bash
sudo systemctl kill nginx
```

Copy code

🔹 4. service vs systemctl

Old way:

powershell

```powershell
service nginx start
```

Copy code

New way:

powershell

```powershell
systemctl start nginx
```

Copy code

👉 Both work, but **systemctl is recommended**

🔹 5. Where service files live

Service config files:

swift

```
/lib/systemd/system/    (system packages)
/etc/systemd/system/    (your custom services)
```

Copy code

You should create custom services in:

swift

```swift
/etc/systemd/system/
```

Copy code

🔹 6. Service File Structure (.service)

Example:

swift

```swift
/etc/systemd/system/myapp.service
```

Copy code

Basic template

ini

```ini
[Unit]
Description=My App Service
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/app/app.py
```

```
WorkingDirectory=/opt/app
Restart=always
User=ubuntu

[Install]
WantedBy=multi-user.target
```

Copy code

Explanation

[Unit]

General info

Option	Meaning
Description	name
After	start after something

[Service]

How to run

Option	Meaning
ExecStart	command
ExecStop	stop command

Option	Meaning
WorkingDirectory	folder
User	run as user
Restart	auto restart
Environment	env variables

[Install]

Boot behavior

Option	Meaning
WantedBy	start at boot

🔹 7. Create Your Own Service (Step-by-Step)

Example: run Python app

Step 1 — create app

bash

```
/opt/app/app.py
```

Copy code

Step 2 — create service file

bash

```bash
sudo nano /etc/systemd/system/myapp.service
```

Copy code

Paste:

ini

```ini
[Unit]
Description=My Python App
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/app/app.py
Restart=always
User=ubuntu

[Install]
WantedBy=multi-user.target
```

Copy code

Step 3 — reload systemd

VERY IMPORTANT:

bash

```bash
sudo systemctl daemon-reload
```

Copy code

Step 4 — start

bash

```bash
sudo systemctl start myapp
```

Copy code

Step 5 — enable boot

bash

```bash
sudo systemctl enable myapp
```

Copy code

Step 6 — check

lua

```lua
systemctl status myapp
```

Copy code

---

8. Logs & Debugging

View logs

nginx

```
journalctl -u nginx
```

Copy code

Follow logs live

powershell

```
journalctl -u nginx -f
```

Copy code

Today only

css

```
journalctl -u nginx --since today
```

Copy code

Last boot

css

```
journalctl -u nginx -b
```

Copy code

---

9. Restart policies (important in production)

Inside `[Service]`

ini

```ini
Restart=always
RestartSec=5
```

Copy code

Options:

value	meaning
no	never
on-failure	only crash
always	always restart

🔹 10. Service Types

ini

```ini
Type=simple
Type=forking
Type=oneshot
Type=notify
```

Copy code

simple (default)

Normal apps

forking

Old daemons

oneshot

Run once then exit

Example:

```nginx
backup job
```

Copy code

🔹 11. Advanced Features

Environment variables

```ini
Environment="PORT=8080"
```

Copy code

or

```ini
EnvironmentFile=/etc/myapp.env
```

Copy code

Limit memory

ini

```ini
MemoryMax=500M
```

Copy code

Limit CPU

ini

```ini
CPUQuota=50%
```

Copy code

Auto restart

ini

```ini
Restart=always
RestartSec=3
```

Copy code

Dependencies

ini

```ini
After=mysql.service
```

Copy code

```
Requires=mysql.service
```

🔹 12. Real Examples

nginx

powershell

```powershell
systemctl start nginx
```

Copy code

docker

bash

```bash
systemctl enable docker
```

Copy code

mysql

nginx

```nginx
systemctl restart mysql
```

Copy code

🔹 13. Useful Quick Commands

Check failed services

```
systemctl --failed
```

Copy code

Reload config

```
systemctl daemon-reload
```

Copy code

Mask service (cannot start)

```
systemctl mask nginx
```

Copy code

Unmask

```
systemctl unmask nginx
```

Copy code

🔹 14. Tips (important)

✅ Always use systemctl

✅ Put custom services in /etc/systemd/system

✅ Run daemon-reload after changes

✅ Use Restart=always for production
✅ Check logs with journalctl

🔥 Quick Cheat Sheet

pgsql

```
start      systemctl start name
stop       systemctl stop name
restart    systemctl restart name
status     systemctl status name
enable     systemctl enable name
disable    systemctl disable name
logs       journalctl -u name -f
reload     systemctl daemon-reload
```

Copy code

If you'd like, next I can teach you:
✅ create NodeJS service
✅ create Spring Boot service
✅ create Docker service
✅ service + nginx + reverse proxy
✅ or deep dive systemd internals

ChatGPT Get Plus × Share •••

explain:
[Unit]
Description=My Python App
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/app/app.py
Restart=always
User=ubuntu

[Install]
WantedBy=multi-user.target

Let’s go **line-by-line** and explain clearly what each part of this **systemd service file** does.

Your file:

ini Copy code

```ini
[Unit]
Description=My Python App
After=network.target
```

```
[Service]
ExecStart=/usr/bin/python3 /opt/app/app.py
Restart=always
User=ubuntu

[Install]
WantedBy=multi-user.target
```

Think of a service file like **3 sections**:

```
[Unit]     → metadata + dependencies
[Service]  → how to run the program
[Install]  → when to start (boot behavior)
```

🔹 [Unit] section → "General info + dependencies"

This section controls:

✅ description
✅ startup order
✅ dependencies between services

1️⃣ Description=My Python App

```
```

Copy code

```
Description=My Python App
```

👉 Just a **human-readable name**

Used when you run:

lua Copy code

```lua
systemctl status myapp
```

You'll see:

Copy code

```
● myapp.service - My Python App
```

No technical effect — only for readability.

2️⃣ After=network.target

ini Copy code

```ini
After=network.target
```

👉 Means:

> Start this service **only AFTER network is ready**

Why needed?

Your Python app might:

- call APIs
- connect database
- open sockets
- use internet

If it starts **before network**, it might crash.

Important:

`After=` controls **order only**, not dependency.

It does NOT guarantee network exists.

If you want strict dependency:

ini Copy code

```ini
Requires=network.target
```

or

ini Copy code

```ini
After=network-online.target
Wants=network-online.target
```

🔹 [Service] section → “How to run program”

This is the **most important part**.

It defines:

- what command to run
- restart behavior
- which user runs it
- environment
- limits

3️⃣ ExecStart=

ruby Copy code

```ruby
ExecStart=/usr/bin/python3 /opt/app/app.py
```

👉 The **command systemd runs**

Equivalent to:

bash Copy code

```bash
python3 /opt/app/app.py
```

Format:

pgsql Copy code

```pgsql
ExecStart=<full path to command>
```

⚠️ Must use FULL PATH (systemd doesn't use your PATH variable)

❌ Wrong:

ini Copy code

```ini
ExecStart=python3 app.py
```

✅ Correct:

ruby Copy code

```ruby
ExecStart=/usr/bin/python3 /opt/app/app.py
```

4️⃣ Restart=always

ini Copy code

```ini
Restart=always
```

👉 Automatically restart if app stops/crashes.

Behavior:

If:

- crash
- exception
- killed
- exit

→ systemd restarts it

Options:

value	meaning
no	never restart
on-failure	restart only crash
always	always restart (best for servers)

Production tip:

Better:

ini Copy code

```ini
Restart=always
RestartSec=3
```

Wait 3s before restart to avoid infinite loop.

5️⃣ User=ubuntu

ini Copy code

```ini
User=ubuntu
```

👉 Run service as this Linux user.

NOT root.

Why important?

Security 🔒

If your app hacked:

- running as root → system compromised
- running as ubuntu → limited damage

Good practice:

Never run apps as root.

You can even create:

bash Copy code

```
sudo useradd myapp
User=myapp
```

🔹 [Install] section → “Boot behavior”

This controls:

👉 When service starts automatically

6️⃣ WantedBy=multi-user.target

ini Copy code

```ini
WantedBy=multi-user.target
```

👉 Start service at **normal system boot**

What is multi-user.target?

Think:

target	meaning
multi-user.target	normal server mode
graphical.target	desktop GUI

target	meaning
rescue.target	safe mode

So:

```ini
WantedBy=multi-user.target
```

means:

👉 start when server fully booted

This works only after:

```bash
systemctl enable myapp
```

Enable creates symlink:

```swift
/etc/systemd/system/multi-user.target.wants/myapp.service
```

🔥 Big Picture Flow

When server boots:

1️⃣ reach multi-user.target
2️⃣ systemd sees myapp wanted
3️⃣ wait for network.target
4️⃣ run python3 app.py as ubuntu
5️⃣ if crash → restart

🔹 Visual Summary

pgsql Copy code

```pgsql
[Unit]
Description → name
After → start order

[Service]
ExecStart → command
Restart → auto restart
User → run as user

[Install]
WantedBy → start at boot
```

🔹 Common Improvements (real-world)

Most production services add:

Example:

ini Copy code

```ini
[Service]
ExecStart=/usr/bin/python3 /opt/app/app.py
WorkingDirectory=/opt/app
Restart=always
RestartSec=5
User=ubuntu
Environment="ENV=prod"
StandardOutput=journal
StandardError=journal
```

Extra features:

- working directory
- restart delay
- env vars
- logs to journal

🔹 Quick Mental Model

Think:

ini

Copy code

```ini
Unit    = when to start
Service = how to run
Install = start at boot or not
```

If you'd like, next I can explain:

✅ all possible Service options
✅ advanced dependencies
✅ oneshot jobs
✅ timers (cron replacement)
✅ create service for Spring Boot / Node / Docker

Just tell me which 👍

how show custom after create

After you **create a custom systemd service**, here's how you can **show / verify / check it properly**.

I'll explain step-by-step what to do and what each command shows.

✅ Step 0 (VERY IMPORTANT)

After creating or editing:

swift Copy code

```swift
/etc/systemd/system/myapp.service
```

You MUST run:

bash Copy code

```bash
sudo systemctl daemon-reload
```

👉 This tells systemd:

"Reload all new/changed service files"

Without this → systemd **won't see your service**.

✅ 1. Show your custom service exists

Method 1 — list all services

bash Copy code

```bash
systemctl list-unit-files --type=service | grep myapp
```

Output example:

Copy code

```
myapp.service    enabled
```

or:

scss Copy code

```scss
myapp.service    disabled
```

Method 2 — check exact file path

lua Copy code

```lua
systemctl status myapp
```

Top line shows:

perl Copy code

```perl
Loaded: loaded (/etc/systemd/system/myapp.service; enabled)
```

👉 confirms:

- file loaded
- path
- enabled or not

✅ 2. Show service status (most useful)

lua Copy code

```lua
systemctl status myapp
```

Shows:

yaml Copy code

```yaml
● myapp.service - My Python App
     Loaded: loaded (/etc/systemd/system/myapp.service; enabled)
     Active: active (running)
   Main PID: 1234
```

You see:

✅ running or stopped
✅ PID
✅ logs
✅ errors

✅ 3. Show service configuration (FULL content)

Show the service file systemd is using

bash Copy code

```
systemctl cat myapp
```

Output:

makefile Copy code

```makefile
# /etc/systemd/system/myapp.service
[Unit]
Description=My Python App
After=network.target
...
```

👉 Best way to confirm:

- correct file
- correct content
- no wrong path

✅ 4. Show dependencies (After / Requires)

If you want to check:

ini Copy code

```ini
After=network.target
```

Show dependency tree:

nginx Copy code

```nginx
systemctl list-dependencies myapp
```

Shows:

pgsql Copy code

```pgsql
myapp.service
● ├─network.target
● ├─system.slice
● └─basic.target
```

👉 confirms it starts AFTER network

✅ 5. Show if enabled at boot

csharp Copy code

```csharp
systemctl is-enabled myapp
```

Output:

nginx Copy code

```nginx
enabled
```

or

scss Copy code

```scss
disabled
```

✅ 6. Show logs

nginx Copy code

```nginx
journalctl -u myapp
```

Live logs:

powershell Copy code

```powershell
journalctl -u myapp -f
```

✅ 7. Show running processes

sql Copy code

```sql
systemctl show myapp
```

Shows **ALL properties** (very detailed):

Example:

ini Copy code

```ini
ExecStart=/usr/bin/python3 /opt/app/app.py
User=ubuntu
Restart=always
```

If you only want specific:

css Copy code

```css
systemctl show myapp -p ExecStart -p User -p Restart
```

🔥 Quick Cheat Sheet

After creating service:

bash Copy code

```bash
sudo systemctl daemon-reload
sudo systemctl enable myapp
sudo systemctl start myapp
```

Check:

powershell Copy code

```powershell
systemctl status myapp
systemctl cat myapp
systemctl list-unit-files | grep myapp
journalctl -u myapp -f
```

✅ Most common workflow (real life)

After create:

lua Copy code

```lua
daemon-reload
start
status
logs
```

Example:

bash Copy code

```bash
sudo systemctl daemon-reload
sudo systemctl start myapp
```

```
systemctl status myapp
journalctl -u myapp -f
```

If you want, next I can teach you:

✅ debug "service not starting"
✅ how to reload without restart
✅ create timer (cron replacement)
✅ advanced dependencies
✅ or how to auto restart only on crash

Just say which 👍

how see log of python server

Since you're running your **Python server with systemd**, the logs are handled by **systemd journal (journald)** automatically.

So you **DON'T check logs like normal files** (unless you redirect them).

✅ Most Important Command (use this first)

Show logs

nginx Copy code

```
journalctl -u myapp
```

👉 Shows **all logs** from your Python service.

✅ Live logs (like tail -f)

powershell Copy code

```
journalctl -u myapp -f
```

👉 Real-time logs (best for debugging)

You'll see:

vbnet Copy code

```
Server started...
Connected to DB
Error: connection timeout
```

Press:

mathematica Copy code

```
Ctrl + C
```

to exit.

✅ Show only today logs

```
journalctl -u myapp --since today
```

✅ Show last 10 minutes

```
journalctl -u myapp --since "10 minutes ago"
```

✅ Show logs from current boot only

```
journalctl -u myapp -b
```

Useful if server restarted.

✅ Show last N lines

nginx Copy code

```
journalctl -u myapp -n 50
```

✅ Show logs with time nicely

cpp Copy code

```
journalctl -u myapp -o short-iso
```

Example:

arduino Copy code

```
2026-02-10T10:22:31 Server started
```

🔹 Why this works automatically?

Because systemd captures:

Python:

lua Copy code

```
print()
logging
errors
traceback
stdout
stderr
```

All go to:

nginx Copy code

```nginx
journald
```

So you don't need:

❌ log files
❌ redirect

🔹 Example

If your app:

python Copy code

```python
print("Server started")

import logging
logging.error("DB failed")
```

You'll see:

nginx Copy code

```nginx
journalctl -u myapp
```

🔹 If logs NOT showing?

Check:

1. Service running?

lua Copy code

```lua
systemctl status myapp
```

2. Python prints flushed?

Sometimes buffer issue. Fix:

Option A (recommended)

Modify service:

ruby Copy code

```ruby
ExecStart=/usr/bin/python3 -u /opt/app/app.py
```

`-u` = unbuffered output

Option B (in Python)

lua Copy code

```lua
print("Hello", flush=True)
```

🔹 Optional: Save logs to file (if you want)

If you prefer file logs:

Method 1 — redirect in service

swift Copy code

```swift
ExecStart=/usr/bin/python3 /opt/app/app.py >> /var/log/myapp.log 2>&1
```

Method 2 — better (Python logging)

Use logging:

python Copy code

```python
logging.basicConfig(
    filename='/var/log/myapp.log',
    level=logging.INFO
)
```

🔥 Recommended (Production)

Use journal:

powershell Copy code

```powershell
journalctl -u myapp -f
```

Because:

✅ automatic
✅ rotated
✅ no disk full
✅ centralized
✅ systemd native

🔹 Quick Cheat Sheet

powershell Copy code

```powershell
journalctl -u myapp          # all logs
journalctl -u myapp -f      # live
journalctl -u myapp -n 100  # last 100
journalctl -u myapp -b      # this boot only
journalctl -u myapp --since today
```

If you'd like, I can next show you:

✅ Python logging best practices

✅ log rotation

✅ send logs to file + journal

✅ or debug “service starts but no logs”

Just tell me 👍

it doesn show anything, while i have tried call it from another server:
root@instance-98480097:~# journalctl -u myapp
-- Logs begin at Fri 2021-09-17 00:26:46 +07, end at Tue 2026-02-10 22:13:20 +07. --
Feb 10 22:12:06 instance-98480097 systemd[1]: Started My Python App.
root@instance-98480097:~#

Good debugging question 👍

If `journalctl -u myapp` **only shows “Started My Python App” but no Python logs**, it means:

👉 **systemd started the service, but your Python stdout/stderr is not reaching journald**

This is very common with Python servers.

Let’s debug step-by-step.

✅ Step 1 — Check if service is REALLY running

First confirm:

lua Copy code

```lua
systemctl status myapp
```

Look for:

yaml Copy code

```yaml
Active: active (running)
Main PID: xxxx
```

If you see:

makefile Copy code

```makefile
Active: exited
```

or

makefile Copy code

```makefile
Active: failed
```

Then your script exited immediately → no logs.

✅ Step 2 — Most common cause: Python output buffering (99% case)

Python buffers output when NOT running in terminal.

So:

bash Copy code

```bash
print("hello")
```

❌ does NOT show immediately in journal.

🔥 Fix (IMPORTANT)

Edit your service:

bash Copy code

```bash
sudo nano /etc/systemd/system/myapp.service
```

Change:

ruby Copy code

```
ExecStart=/usr/bin/python3 /opt/app/app.py
```

👉 to:

ruby Copy code

```ruby
ExecStart=/usr/bin/python3 -u /opt/app/app.py
```

`-u` **means:**

lua Copy code

```lua
unbuffered output
```

Forces:

- print()
- logging
- errors

→ immediately sent to journald

Then reload + restart

bash Copy code

```bash
sudo systemctl daemon-reload
sudo systemctl restart myapp
```

Now test:

powershell Copy code

```powershell
journalctl -u myapp -f
```

✅ Step 3 — Ensure logs go to journal

Add these (recommended):

ini Copy code

```ini
StandardOutput=journal
StandardError=journal
```

Your service should look like:

ini Copy code

```ini
[Service]
ExecStart=/usr/bin/python3 -u /opt/app/app.py
Restart=always
User=ubuntu
StandardOutput=journal
StandardError=journal
```

✅ Step 4 — Test quickly

Add this to your python:

python Copy code

```python
import time

print("Server started", flush=True)

while True:
    print("alive", flush=True)
    time.sleep(5)
```

Then:

powershell Copy code

```powershell
journalctl -u myapp -f
```

You should see:

arduino Copy code

```arduino
Server started
alive
alive
alive
```

✅ Step 5 — If still nothing → check these

Check 1 — maybe your app daemonizes/forks?

If your app uses:

- gunicorn
- uvicorn
- flask production server
- multiprocessing
- background fork

Then systemd might lose stdout.

Fix:

Use:

ini Copy code

```ini
Type=simple
```

or avoid daemon mode.

Example:

css Copy code

```
gunicorn --daemon    ❌ BAD
gunicorn             ✅ GOOD
```

Systemd should manage the daemon, not the app.

Check 2 — manually run command

Run EXACT same command:

bash Copy code

```bash
sudo -u ubuntu /usr/bin/python3 -u /opt/app/app.py
```

If nothing prints → your code isn't printing.

Check 3 — confirm correct service file

bash Copy code

```bash
systemctl cat myapp
```

Ensure it's loading the file you edited.

Sometimes:

- wrong filename
- wrong directory
- forgot daemon-reload

🔥 Final Recommended Service (best practice)

Use this:

ini Copy code

```ini
[Unit]
Description=My Python App
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 -u /opt/app/app.py
WorkingDirectory=/opt/app
Restart=always
RestartSec=3
User=ubuntu
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

✅ 99% fix checklist

Do these exactly:

pgsql Copy code

```pgsql
add -u
add StandardOutput/StandardError
daemon-reload
restart
journalctl -f
```

If still not working, paste:

bash Copy code

```bash
systemctl status myapp
systemctl cat myapp
```

and I'll debug with you 👍